# Lab06

Max Stróżyk

Winter 2024/2025

English translation of the Polish original. Historical results and formulas retained.

## Contents

**1 Task 1: Newton–Cotes quadrature** **2**
1.1 Method overview and problem . . . 2
1.1.1 Detailed explanation of the formula . . . 2
1.2 Tests . . . 2
1.3 Conclusions and analysis . . . 4

**2 Task 2: analysis of numerical methods** **6**
2.1 Method overview and problem . . . 6
2.1.1 Bisection method . . . 6
2.1.2 False-position method . . . 6
2.1.3 Newton–Raphson method . . . 6
2.1.4 Secant method . . . 7
2.2 Tests . . . 7
2.3 Conclusions and analysis . . . 10

**Project on GitHub**

# 1 Task 1: Newton–Cotes quadrature

## 1.1 Method overview and problem

Composite Newton–Cotes quadrature integrates functions using the Newton–Cotes formula over composite intervals. It aims to improve integration precision, particularly for more complex functions. **Assumptions:**

- Function $f$: the function to integrate.
- Endpoints $a$ and $b$: integration limits.
- Point count $n$: number of interpolation points.
- Interval count $m$: number of subintervals.

**Formula:**

$$\int_a^b f(x)dx \approx \frac{b-a}{mn} \sum_{k=0}^{n} B_k \sum_{s=0}^{m-1} f(a + \frac{(ns+k)(b-a)}{mn})$$

The coefficients $B_k$ are computed using Lagrange basis polynomials. **Key properties:**

- Multiple subintervals $m$ reduce the Runge phenomenon.
- Higher accuracy than simple Newton–Cotes formulas at the same point count.
- Applicable to both uniformly and nonuniformly continuous functions.

### 1.1.1 Detailed explanation of the formula

The method computes an integral using polynomial approximation. We fix the polynomial degree ($m$), then evaluate each integral $\int_a^b P_i$, where $P_i$ is the polynomial on subinterval $i$. Computing the integral on subinterval $i$ requires $m$ points, so the full approximation needs $n * m$ points; I did not account for this here, which is my mistake. To find $P_i$, we use Lagrange interpolation and integrate those polynomials. I did not implement a function for this either, although integrating polynomials is not difficult.

## 1.2 Tests

Tests validated the method and analyzed its errors. They included integrating $x^2$ over $[-1, 6]$ and comparing the result with the analytical solution. **Error analysis:**

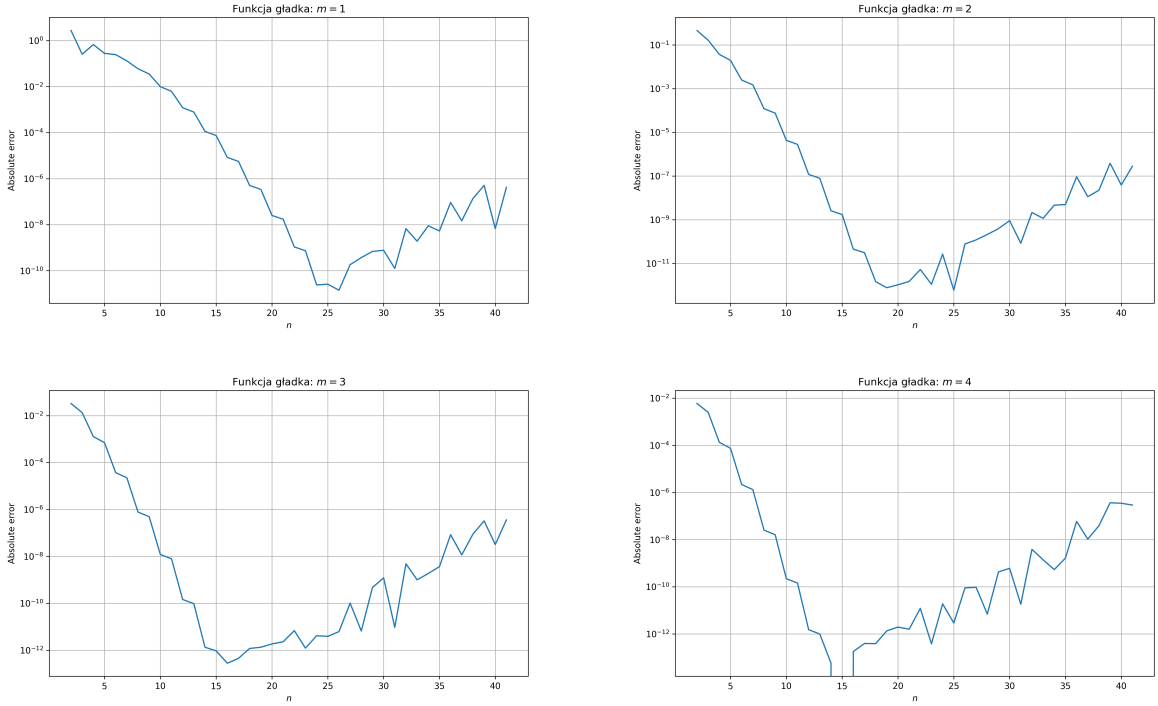
- We compared performance for two test functions:
  - Uniformly continuous function: $\sin(x) + \cos^2(x) - x^2 + 7x$
  - Nonuniformly continuous function: $\frac{1}{1+x^2}$

For each function we examined:

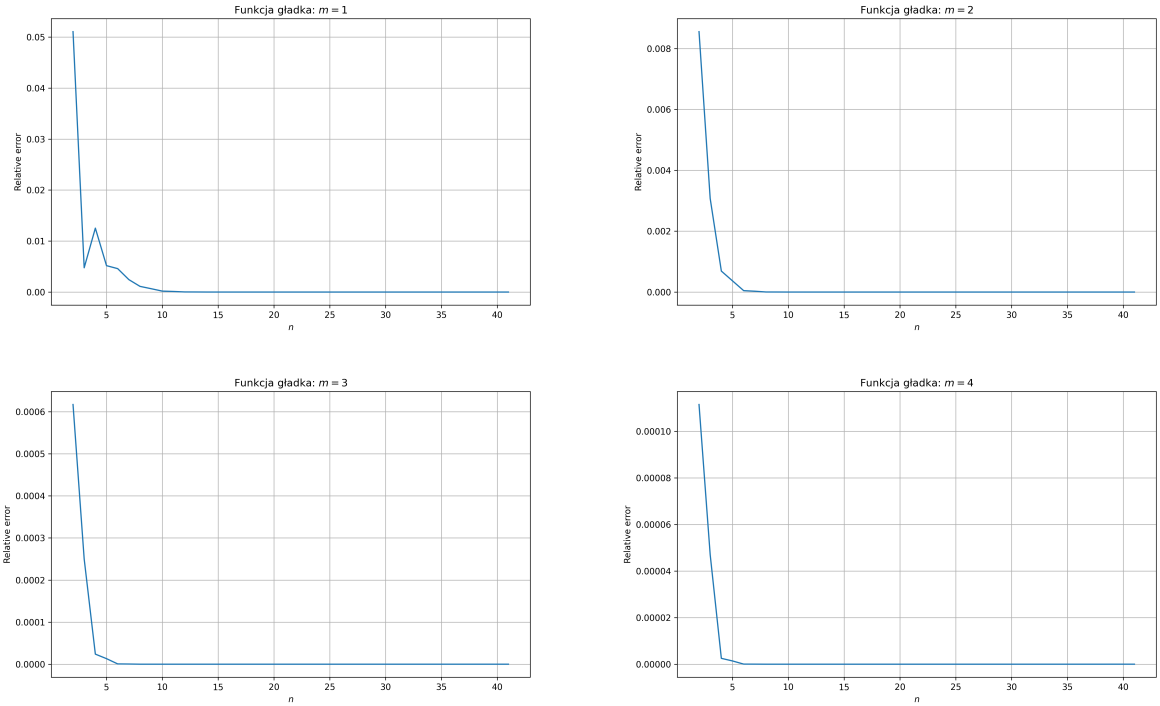
- Relative and absolute errors with identical parameter settings.
- Parameter ranges:
  - $m$: from 1 to 4, the number of subintervals.
  - $n$: from 2 to 42, the number of points.

## Error growth

### Uniformly continuous function

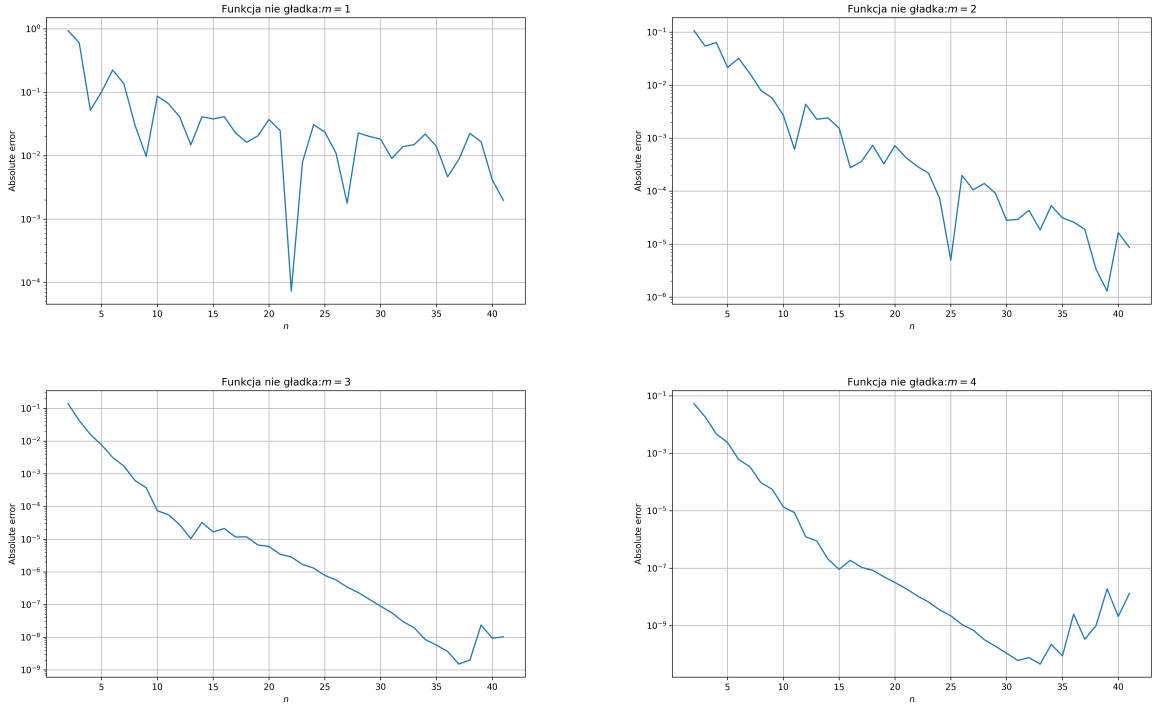



Relative error

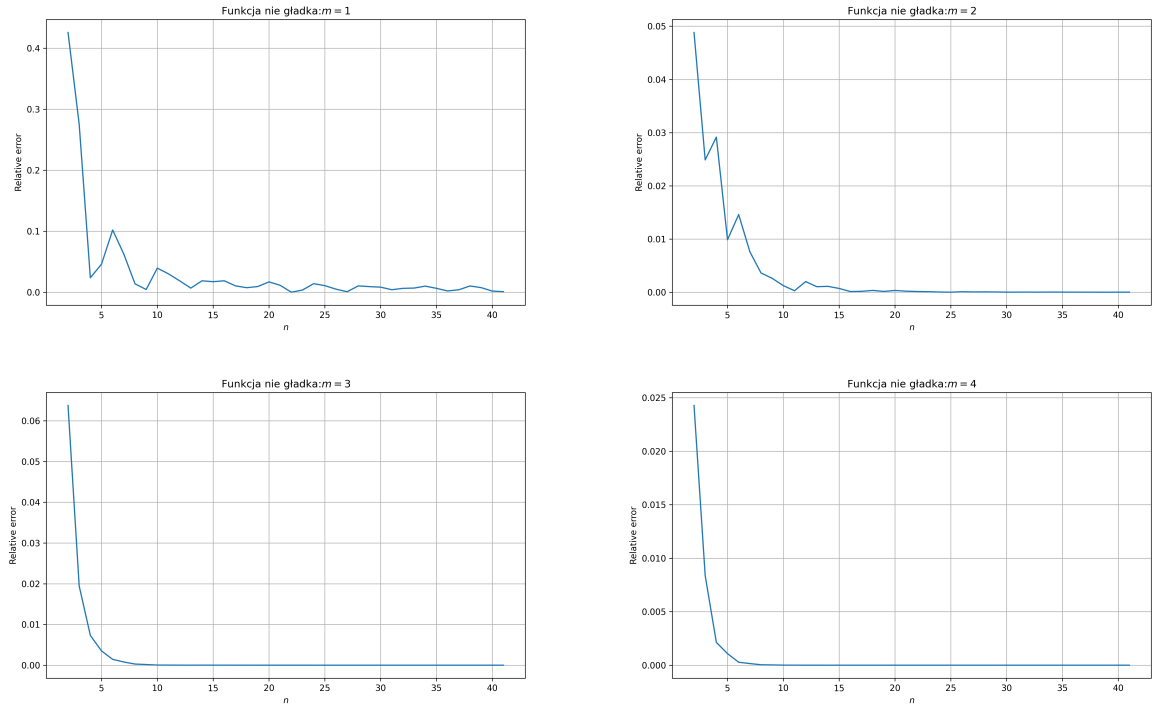



Absolute error

### Nonuniformly continuous function




Relative error




Absolute error

## 1.3 Conclusions and analysis

The tests support the following conclusions:

- Increasing $m$, the formula order, drastically reduces the error. This is clearest for the function that is not uniformly continuous. Between 2 and 3, the error falls almost to zero, allowing accurate results quickly.

- High accuracy is attained quickly. However, too many intervals lead to the Runge phenomenon: the error starts increasing.
- The method could be optimized further. Compared with the `quad` method from `scipy.integrate`, it is the most accurate of the basic numerical methods.

# 2 Task 2: analysis of numerical methods

## 2.1 Method overview and problem

This task analyzes bisection, false position, Newton–Raphson and secant methods for finding function roots. Each is described in terms of assumptions, formula derivation and implementation.

### 2.1.1 Bisection method

Bisection is one of the simplest numerical root-finding methods.

- Assumptions:
  - $f(x)$ is continuous on $[a, b]$.
  - At least one root exists in $[a, b]$, that is, $f(a) \cdot f(b) < 0$.
- Formula:
$$c = \frac{a + b}{2}$$
- Derivation: Repeatedly bisect $[a, b]$. At each step, select the subinterval containing a root using the sign of $f(a) \cdot f(c)$. Continue until the desired accuracy acc is reached.

### 2.1.2 False-position method

False position uses a line to find roots from function values at interval endpoints.

- Assumptions:
  - $f(x)$ is continuous on $[z_1, z_2]$.
  - $f(z_1) \cdot f(z_2) < 0$.
- Formula:
$$c = \frac{a \cdot f(b) - b \cdot f(a)}{f(b) - f(a)}$$
- Derivation: Approximate a root using the line through $(a, f(a))$ and $(b, f(b))$. The next point $c$ is its intersection with the $x$-axis ($f(c) = 0$).
$$\frac{-f(a)}{c - a} = \frac{f(c) - f(a)}{c - a} = \frac{f(b) - f(a)}{b - a}$$
Rearrangement gives:
$$c = \frac{a \cdot f(b) - b \cdot f(a)}{f(b) - f(a)}$$
Repeat until the difference between successive approximations reaches the desired accuracy.

### 2.1.3 Newton–Raphson method

Newton–Raphson is a fast iterative method using function derivatives.

- Assumptions:
  - $f(x)$ is differentiable near the root.
  - A starting point $x_0$ near the root is known.

- Formula:
$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)}$$
- Derivation: Expand the function in a Taylor series at $x_i$.
$$f(x) \sim f(x_0) + f'(x_0)(x - x_0)$$
Rearranging and assuming $(f(x) = 0)$ gives:
$$x = x_0 - \frac{f(x_0)}{f'(x_0)}$$
The tangent at $x_i$ intersects the $x$-axis at the next approximation $x_{i+1}$. Iterate until $|f(x_{i+1})| <$ acc.

### 2.1.4 Secant method

The secant method modifies Newton–Raphson so that derivatives need not be computed.

- Assumptions:
  - $f(x)$ need not be differentiable.
  - Two starting points $x_0$ and $x_1$ are known.
- Formula:
$$x_{i+1} = x_i - \frac{f(x_i)(x_i - x_{i-1})}{f(x_i) - f(x_{i-1})}$$
- Derivation: Replace the tangent with the secant through successive points $(x_{i-1}, f(x_{i-1}))$ and $(x_i, f(x_i))$. Expand the derivative using its definition and substitute the corresponding values.

## 2.2 Tests

Two test types were performed for each method:

- Error versus iteration count, for ranges (2-20 and 2-120).
- Iteration count and runtime needed for a specified accuracy $\in [10^{-1}, 10^{-32}$. Maximum iteration count = 1000.

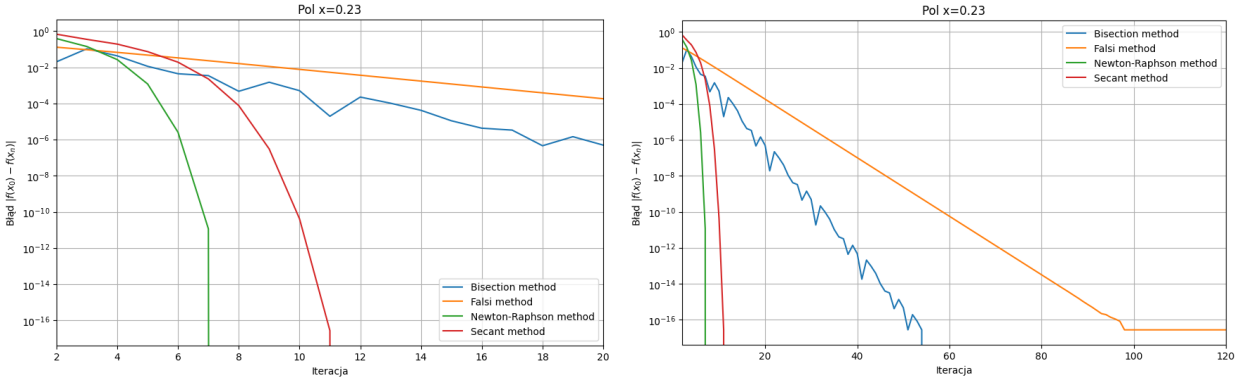
Test functions:

- Pol $x = 0.23$: $f(x) = (x - 0.23)(x + 1)(x + 2)(x + 3)(x + 4)$
- Pol $x = 0.53$: $f(x) = (x - 0.53)(x + 1)(x + 2)(x + 3)(x + 4)$

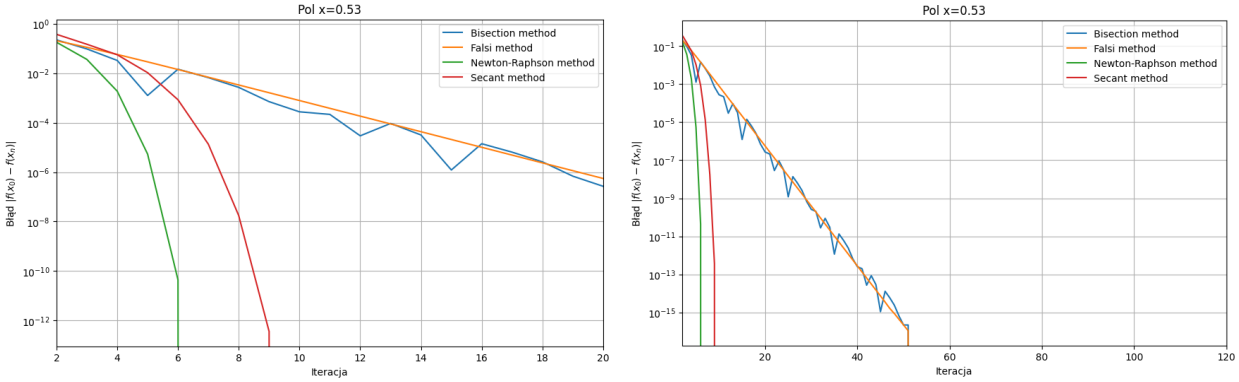
For each function, we considered starting points nearer the center of the interval and nearer an endpoint.

**Error growth**

**Interval** $[0, 1]$

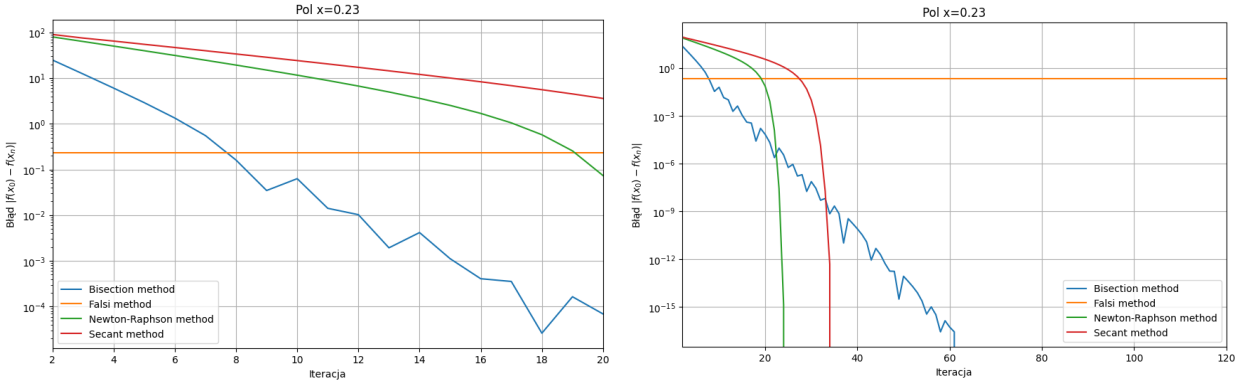



Polynomial with $x_0 = 0.23$

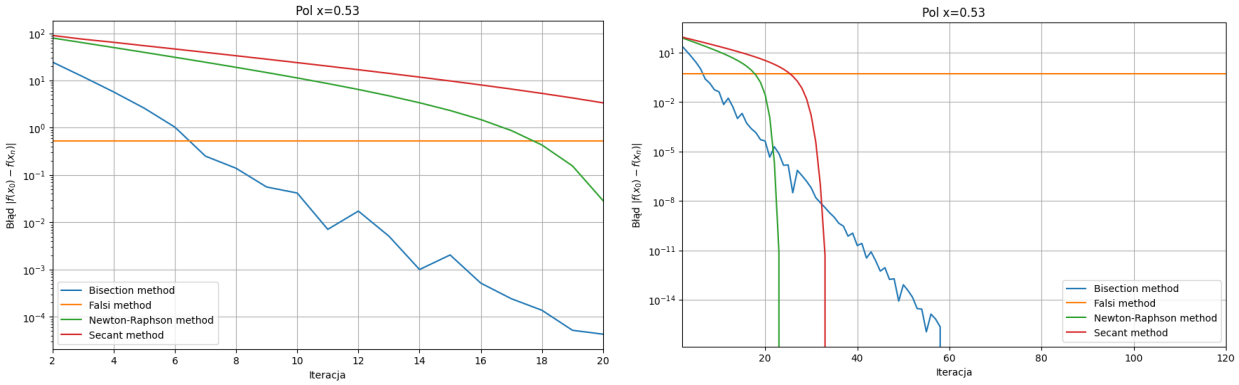



Polynomial with $x_0 = 0.53$

**Interval** $[0, 100]$



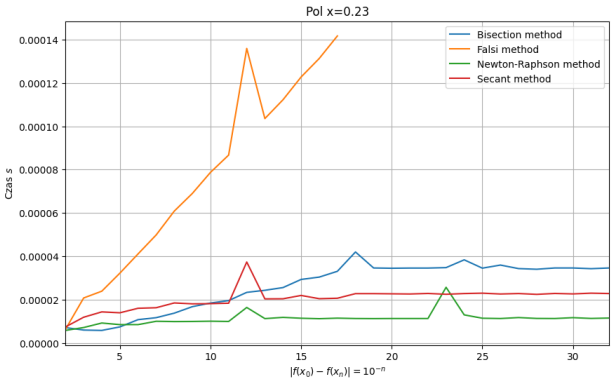

Polynomial with $x_0 = 0.23$

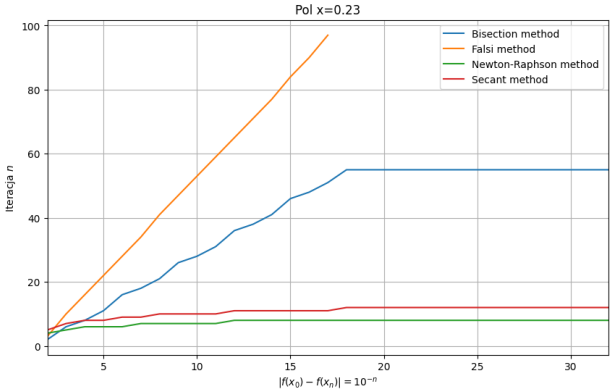



Polynomial with $x_0 = 0.53$

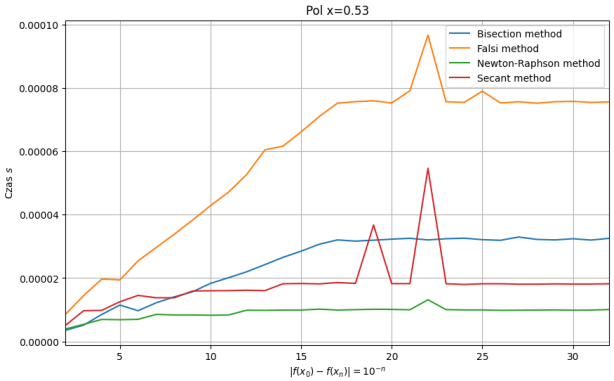
### Attaining the required accuracy

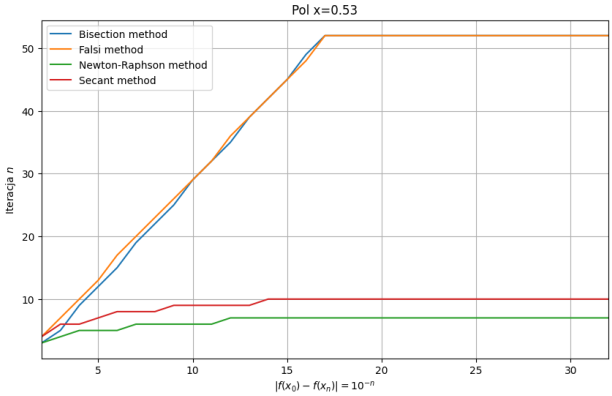
**Interval** $[0, 1]$



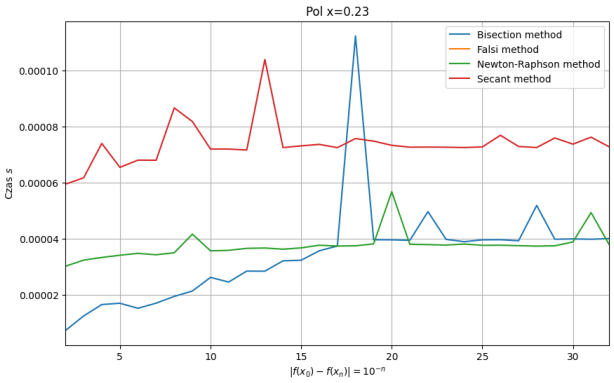

Polynomial with $x_0 = 0.23$

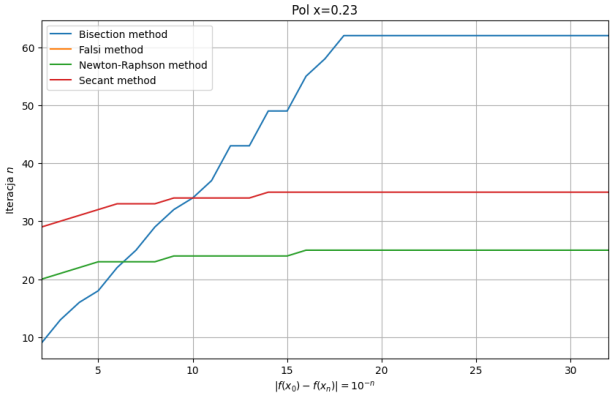


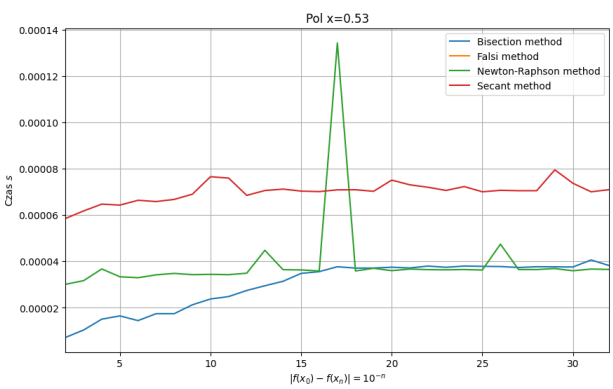

Polynomial with $x_0 = 0.53$

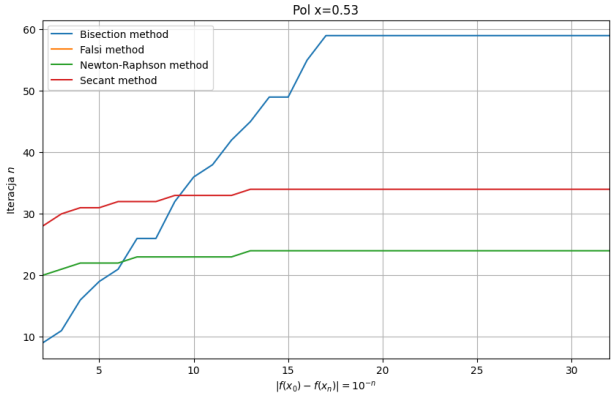
**Interval** $[0, 100]$




Polynomial with $x_0 = 0.23$




Polynomial with $x_0 = 0.53$

## 2.3 Conclusions and analysis

### Error versus iteration count

- Newton–Raphson is fastest, although it requires an initial point near the root. Its error decreases exponentially, making it very efficient.
- The secant method is slower than Newton–Raphson but permits approximation of nondifferentiable functions. Its accuracy also increases exponentially, with a smaller convergence coefficient.
- False position reduces error linearly but is ineffective for roots extremely close to an endpoint. Its runtime is worse than or comparable with bisection. The test functions may have been particularly unsuitable for this method, which proposes a statistical solution and should be more efficient.

### Iterations and runtime needed to reach the target precision

- For bisection and false position, runtime and iteration count grow linearly with the required precision.
- For Newton–Raphson and the secant method, iteration count and runtime are constant once the root is reached.
- Over a large interval, false position was inefficient and sometimes did not converge, making it unusable in those cases.